

Operating Systems: Assignment-12

Name: Suraj Shah

Admission Number: I24AI012

Q Objective

To implement and compare different memory allocation algorithms:

1. First Fit
2. Next Fit
3. Best Fit
4. Worst Fit
5. Quick Fit

Problem Statement

Simulate dynamic memory allocation in an operating system using different strategies. Analyze their performance in terms of:

- Memory utilization
- Fragmentation (internal & external)
- Allocation speed

Tasks

1. Implement the Following Algorithms

Write a C program that implements:

1. First Fit – Allocates the first available block that is large enough.
2. Next Fit – Similar to First Fit but starts searching from the last allocated block.
3. Best Fit – Allocates the smallest block that is large enough.
4. Worst Fit – Allocates the largest available block.
5. Quick Fit – Maintains separate free lists for common block sizes.

2. Simulation & Analysis

- Generate random process sizes (e.g., 16KB–256KB).
- Simulate allocation and deallocation.

- Track:
 - o Success rate (percentage of successful allocations)
 - o Memory utilization (used vs. free memory)
 - o Fragmentation (wasted memory due to small gaps)
 - o Execution time (speed of allocation)

3. Comparison & Report

- Compare the algorithms based on the collected metrics.
- Discuss which strategy performs best under different scenarios.

Lab Requirements

1. Program Structure

- Use a linked list to represent memory blocks.
- Implement functions for:
 - o `initialize_memory()`
 - o `allocate_memory(strategy, size)`
 - o `free_memory(process_id)`
 - o `display_memory()`
 - o `coalesce_free_blocks()` (to merge adjacent free blocks)

2. Expected Output

- Memory status after each allocation/deallocation.
- Statistics (success rate, fragmentation, time taken).

3. Sample Test Cases

- Test Case 1: Allocate 5 processes (e.g., 64KB, 128KB, 32KB, 256KB, 16KB).
- Test Case 2: Free some processes and reallocate.
- Test Case 3: Stress test with 20+ random allocations.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MEMORY_SIZE 1024
#define MAX_PROCESSES 100

typedef struct Block {
    int size;
    int free;
    int pid;
    struct Block* next;
} Block;

Block* head=NULL;
int last_alloc=0;
int pid_counter=1;

void initialize_memory() {
    head=(Block*)malloc(sizeof(Block));
    head->size=MEMORY_SIZE;
    head->free=1;
    head->pid=-1;
    head->next=NULL;
}

void display_memory() {
    Block* temp=head;
    printf("\nMemory State:\n");
    while (temp) {
        printf("[Size: %d KB | %s | PID: %d] -> ", temp->size, temp->free ?
"Free" : "Used", temp->pid);
        temp=temp->next;
    }
    printf("NULL\n");
}

void split(Block* fit, int size) {
    Block* newBlock=(Block*)malloc(sizeof(Block));
    newBlock->size=fit->size-size;
    newBlock->free=1;
    newBlock->pid=-1;
    newBlock->next=fit->next;

    fit->size=size;
    fit->free=0;
    fit->pid=pid_counter++;
}
```

```

    fit->next=newBlock;
}

void first_fit(int size) {
    Block* temp=head;

    while (temp) {
        if (temp->free && temp->size>=size) {
            split(temp, size);
            return;
        }
        temp=temp->next;
    }
    printf("First Fit: Allocation failed\n");
}

void next_fit(int size) {
    static Block* last=NULL;
    if (!last) {
        last=head;
    }

    Block* start=last;
    do {
        if (last->free && last->size>=size) {
            split(last, size);
            return;
        }
        last=last->next ? last->next : head;
    } while (last!=start);

    printf("Next Fit: Allocation failed\n");
}

void best_fit(int size) {
    Block *temp=head, *best=NULL;

    while (temp) {
        if (temp->free && temp->size>=size) {
            if (!best || temp->size<best->size) {
                best=temp;
            }
        }
        temp=temp->next;
    }

    if (best) {
        split(best, size);
    }
}

```

```

    }
    else {
        printf("Best Fit: Allocation failed\n");
    }
}

void worst_fit(int size) {
    Block *temp=head, *worst=NULL;

    while (temp) {
        if (temp->free && temp->size>=size) {
            if (!worst || temp->size>worst->size) {
                worst=temp;
            }
        }
        temp=temp->next;
    }

    if (worst) {
        split(worst, size);
    }
    else {
        printf("Worst Fit: Allocation failed\n");
    }
}

#define QUICK_CLASSES 3
int quick_size[QUICK_CLASSES]={32, 64, 128};

void quick_fit(int size) {
    Block* temp=head;

    while (temp) {
        if (temp->free && temp->size>=size) {
            split(temp, size);
            return;
        }
        temp=temp->next;
    }
    printf("Quick Fit: Allocation failed\n");
}

void free_memory(int pid) {
    Block* temp=head;

    while (temp) {
        if(!temp->free && temp->pid==pid) {
            temp->free=1;

```

```

        temp->pid=-1;
        printf("Process is %d freed\n", pid);
        return;
    }
    temp=temp->next;
}
printf("PID not found\n");
}

void coalesce() {
    Block* temp=head;

    while (temp && temp->next) {
        if(temp->free && temp->next->free) {
            temp->size+=temp->next->size;
            temp->next=temp->next->next;
        }
        else {
            temp=temp->next;
        }
    }
}

void statistics() {
    Block* temp=head;
    int used=0, free=0, fragments=0;

    while (temp) {
        if (temp->free) {
            free+=temp->size;
            fragments++;
        }
        else {
            used+=temp->size;
        }
        temp=temp->next;
    }

    printf("\nStatistics\n");
    printf("Used Memory: %d KB\n", used);
    printf("Free Memory: %d KB\n", free);
    printf("Fragments: %d\n", fragments);
    printf("Utilization: %.2f%%\n", (used*100.0)/MEMORY_SIZE);
}

int main() {
    initialize_memory();

```

```
int choice, size, pid;

while (1) {
    printf("\n1.First Fit\n2.Next Fit\n3.Best Fit\n4.Worst Fit\n5.Quick  
Fit\n6.Free\n7.Display\n8.Exit\n9.Stress Test\nChoice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter size: ");
            scanf("%d", &size);
            first_fit(size);
            break;

        case 2:
            printf("Enter size: ");
            scanf("%d", &size);
            next_fit(size);
            break;

        case 3:
            printf("Enter size: ");
            scanf("%d", &size);
            best_fit(size);
            break;

        case 4:
            printf("Enter size: ");
            scanf("%d", &size);
            worst_fit(size);
            break;

        case 5:
            printf("Enter size: ");
            scanf("%d", &size);
            quick_fit(size);
            break;

        case 6:
            printf("Enter PID: ");
            scanf("%d", &pid);
            free_memory(pid);
            coalesce();
            break;

        case 7:
            display_memory();
            statistics();
    }
}
```

```
        break;

    case 8:
        exit(0);
    case 9:
        for (int i=0; i<20; i++) {
            int size=rand()%240+16;
            printf("\nAllocating %d KB\n", size);
            first_fit(size);
        }
        break;
    }
}
```


Output:

```
1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 1
Enter size: 64
```

```
1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 1
Enter size: 128
```

```
1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 1
Enter size: 32
```

```
1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
```

```
8.Exit
Choice: 1
Enter size: 256

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 1
Enter size: 16

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 7

Memory State:
[Size: 64 KB | Used | PID: 1] -> [Size: 128 KB | Used | PID: 2] -> [Size: 32 KB | Used | PID: 3] -> [Size: 256 KB | Used | PID: 4] -> [Size: 16 KB | Used | PID: 5] -> [Size: 528 KB | Free | PID: -1] -> NULL

Statistics
Used Memory: 496 KB
Free Memory: 528 KB
Fragments: 1
Utilization: 48.44%

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit

6.Free
7.Display
8.Exit
Choice: 6
Enter PID: 2
Process is 2 freed

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 7

Memory State:
[Size: 64 KB | Used | PID: 1] -> [Size: 128 KB | Free | PID: -1] -> [Size: 32 KB | Used | PID: 3] -> [Size: 256 KB | Used | PID: 4] -> [Size: 16 KB | Used | PID: 5] -> [Size: 528 KB | Free | PID: -1] -> NULL

Statistics
Used Memory: 368 KB
Free Memory: 656 KB
Fragments: 2
Utilization: 35.94%

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 1
Enter size: 100

1.First Fit
2.Next Fit

3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
Choice: 7

Memory State:
[Size: 64 KB | Used | PID: 1] -> [Size: 100 KB | Used | PID: 6] -> [Size: 28 KB | Free | PID: -1] -> [Size: 32 KB | Used | PID: 3] -> [Size: 256 KB | Used | PID: 4] -> [Size: 16 KB | Used | PID: 5] -> [Size: 528 KB | Free | PID: -1] -> NULL

Statistics
Used Memory: 468 KB
Free Memory: 556 KB
Fragments: 2
Utilization: 45.76%
```

```
1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
9.Stress Test
Choice: 9
```

```
Allocating 57 KB
```

```
Allocating 243 KB
```

```
Allocating 110 KB
```

```
Allocating 116 KB
```

```
Allocating 225 KB
```

```
Allocating 140 KB
```

```
Allocating 214 KB
First Fit: Allocation failed
```

```
Allocating 94 KB
```

```
Allocating 98 KB
First Fit: Allocation failed
```

```
Allocating 240 KB
First Fit: Allocation failed
```

```
Allocating 201 KB
First Fit: Allocation failed
```

```
Allocating 81 KB
First Fit: Allocation failed
```

```
Allocating 17 KB

Allocating 43 KB
First Fit: Allocation failed

Allocating 137 KB
First Fit: Allocation failed

Allocating 27 KB
First Fit: Allocation failed

Allocating 131 KB
First Fit: Allocation failed

Allocating 198 KB
First Fit: Allocation failed

Allocating 43 KB
First Fit: Allocation failed

Allocating 172 KB
First Fit: Allocation failed

1.First Fit
2.Next Fit
3.Best Fit
4.Worst Fit
5.Quick Fit
6.Free
7.Display
8.Exit
9.Stress Test
Choice: 7

Memory State:
[Size: 57 KB | Used | PID: 1] -> [Size: 243 KB | Used | PID: 2] -> [Size: 110 KB | Used | PID: 3] -> [Size: 116 KB | Used | PID: 4] -> [Size: 225 KB | Used | PID: 5] -> [Size: 140 KB | Used | PID: 6] -> [Size: 94 KB | Used | PID: 7] -> [Size: 17 KB | Used | PID: 8] -> [Size: 22 KB | Free | PID: -1] -> NULL

Statistics
Used Memory: 1002 KB
```

Free Memory: 22 KB
Fragments: 1
Utilization: 97.85%